

Documentation for Leetcode 565: Array Nesting

Table of Contents

1. [Problem Statement](#)
2. [Intuition](#)
3. [Key Observations](#)
4. [Approach](#)
5. [Edge Cases](#)
6. [Complexity Analysis](#)
 - o Time Complexity
 - o Space Complexity
7. [Alternative Approaches](#)
8. [Algorithm](#)
9. [Test Cases](#)
10. [Final Thoughts](#)

1. Problem Statement

Given an array `nums` of length `n` where `nums` is a permutation of $[0, n-1]$, construct a set `s[k]` starting at index `k` defined as:

$$s[k] = \{nums[k], nums[nums[k]], nums[nums[nums[k]]], \dots\}$$

Stop when an element repeats (i.e., when the next value has already been added). Return the maximum size among all such sets.

2. Intuition

Since every element is unique and the array is a permutation of 0 to `n-1`, the sets formed will be cyclic. Once we enter a cycle, we can count its length and mark elements as visited to avoid re-processing them.

3. 🔍 Key Observations

- Every value appears exactly once (permutation).
- Each $s[k]$ defines a cycle.
- Visiting an already seen index will result in a duplicate set — we should skip those.
- We only need to visit each index once.

4. ☐ Approach

- Create a visited array to track whether an index has been processed.
- Loop through each index:
 - If not visited, follow the chain (i.e., keep applying $\text{nums}[i]$).
 - Count the number of steps until the cycle completes.
 - Update max_len with the largest count.
- Return the max_len .

5. ✨ Edge Cases

- $\text{nums} = [0]$ → Only one element; cycle length = 1.
- $\text{nums} = [1, 0]$ → Two elements forming a cycle; cycle length = 2.
- $\text{nums} = [0, 1, 2]$ → Each element points to itself; all cycle lengths = 1.

6. ☐ Complexity Analysis

☐ Time Complexity

- $O(n)$ — Each index is visited at most once.

📦 Space Complexity

- $O(n)$ — For the visited array.

7. 🔄 Alternative Approaches

a. In-Place Modification

- Set `nums[i] = -1` after visiting.
- Reduces space complexity to $O(1)$.
- Only works if you are allowed to modify the input.

8. □ Algorithm

- Initialize `visited = [False] * n`, `max_len = 0`.
- For each index `i` in `nums`:
 - If `visited[i]` is `False`:
 - Start from `i`.
 - While the current index is not visited:
 - ✓ Mark it as visited.
 - ✓ Move to `nums[current]`.
 - ✓ Increment count.
 - Update `max_len` if count is greater.
- Return `max_len`.

9. □ Test Cases

Input	Output	Explanation
<code>[5,4,0,3,1,6,2]</code>	4	Cycle: $0 \rightarrow 5 \rightarrow 6 \rightarrow 2 \rightarrow 0$
<code>[0]</code>	1	Single element loop
<code>[1,0]</code>	2	Cycle: $0 \rightarrow 1 \rightarrow 0$
<code>[2,0,1,4,5,3]</code>	3	Two cycles: $0 \rightarrow 2 \rightarrow 1 \rightarrow 0$ and $3 \rightarrow 4 \rightarrow 5 \rightarrow 3$
<code>[0,1,2,3,4]</code>	1	Each element points to itself

10. □ Final Thoughts

- This is a cycle detection problem on a permutation.
- Visited tracking ensures each node is only processed once.
- You can optimize space by modifying the input (if allowed).
- A good example of applying graph traversal (like DFS) on arrays.